

*String formatting*, or *string interpolation*, is the process of creating strings that include other strings and has had a long history in Python. Originally, the `+` operator could concatenate strings together, but this resulted in code with many quotes and pluses: `'Hello, ' + name + '. Today is ' + day + ' and it is ' + weather + '.'`. The `%s` conversion specifier made the syntax a bit easier: `'Hello, %s. Today is %s and it is %s.' % (name, day, weather)`. Both techniques will insert the strings in the `name`, `day`, and `weather` variables into the string literals to evaluate to a new string value, like this: `'Hello, Al. Today is Sunday and it is sunny.'`

The `format()` string method adds the *Format Specification Mini-Language* (<https://docs.python.org/3/library/string.html#formatspec>), which involves using `{}` brace pairs in a way similar to the `%s` conversion specifier. However, the method is somewhat convoluted and can produce unreadable code, so I discourage its use.

But as of Python 3.6, *f-strings* (short for *format strings*) offer a more convenient way to create strings that include other strings. Just like how raw strings are prefixed with an `r` before the first quote, f-strings are prefixed with an `f`. You can include variable names in between braces in the f-string to insert the strings stored in those variables:

---

```
>>> name, day, weather = 'Al', 'Sunday', 'sunny'
>>> f'Hello, {name}. Today is {day} and it is {weather}.'
'Hello, Al. Today is Sunday and it is sunny.'
```

---

The braces can contain entire expressions as well:

---

```
>>> width, length = 10, 12
>>> f'A {width} by {length} room has an area of {width * length}.'
'A 10 by 12 room has an area of 120.'
```

---

If you need to use a literal brace inside an f-string, you can escape it with an additional brace:

---

```
>>> spam = 42
>>> f'This prints the value in spam: {spam}'
'This prints the value in spam: 42'
>>> f'This prints literal curly braces: {{spam}}'
'This prints literal curly braces: {spam}'
```

---